Atividade Avaliativa - 3º Trimestre

Tutorial: Pipeline de Dados com a API NASA POWER

Coleta automatizada com GitHub Actions, armazenamento no Supabase (PostgreSQL) e painel web no GitHub Pages

Grupo 15

Jakson José e Nayro Ganzer

Repositório: https://github.com/SN-2026-GRUPO-15-NASA/POWER

Painel publicado: https://sn-2026-grupo-15-nasa.github.io/POWER/

Novembro de 2026

1. Objetivo e resultado final

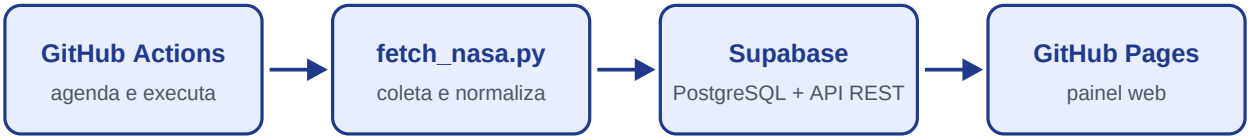
Este guia explica, passo a passo, como reproduzir o projeto do grupo: um pipeline de dados que busca dados climatológicos diários na API aberta NASA POWER, grava os registros em um banco PostgreSQL no Supabase e os exibe em um painel web publicado no GitHub Pages. A coleta roda sozinha todos os dias e também pode ser disparada manualmente.

Ao final, a pessoa que seguir o tutorial terá:

- um repositório público no GitHub com o script de coleta, o workflow e o painel;
- um banco no Supabase com duas tabelas (dados e log de execuções), com segurança por linha (RLS) ativa;
- um painel com filtros de cidade, variável e período, gráficos, ranking, tabela ordenável e a data da última atualização;
- pelo menos uma execução do workflow com ícone verde e dados reais gravados.

Tempo estimado: de 1 a 2 horas, sem custo (planos gratuitos do GitHub e do Supabase).

2. Visão geral da arquitetura



NASA POWER (API) alimenta o script; o painel le os dados pela API REST do Supabase

- **GitHub Actions:** executa o workflow todos os dias às 06:00 UTC (03:00 em Brasília) e quando alguém clica em Run workflow.
- **Script fetch_nasa.py:** consulta a API NASA POWER para cada cidade, converte os dados, elimina duplicatas e envia ao Supabase em lotes.
- **Supabase:** guarda os dados na tabela `clima_diario`, registra cada execução em `execucoes` e expõe uma API REST automática.
- **GitHub Pages:** hospeda o painel (HTML e JavaScript), que lê a API REST do Supabase usando apenas a chave pública.

A API NASA POWER

A NASA POWER (Prediction Of Worldwide Energy Resources) fornece dados meteorológicos e de energia solar para qualquer coordenada do planeta. A API não exige chave. O endpoint usado é o de dados diários por ponto:

```
https://power.larc.nasa.gov/api/temporal/daily/point
```

Parâmetro	Coluna no banco	Significado
T2M	temp_media	Temperatura do ar a 2 m (graus Celsius)
PRECTOTCORR	precipitacao	Precipitação corrigida (mm por dia)
ALLSKY_SFC_SW_DWN	radiacao_solar	Radiação solar na superfície (kWh/m^2 por dia)
WS2M	vento	Velocidade do vento a 2 m (m/s)

Parâmetro	Coluna no banco	Significado
RH2M	umidade	Umidade relativa (%)

Particularidades: valores ausentes vêm como **-999** (o script grava como NULL); os dados mais recentes têm alguns dias de atraso (o script termina três dias antes de hoje); e cada chamada é feita para uma única coordenada, por isso o script percorre uma lista de cidades.

3. Pré-requisitos

- Uma conta no GitHub (github.com) e uma conta no Supabase (supabase.com), ambas gratuitas.
- Um navegador atualizado. Não é necessário instalar nada no computador: todos os arquivos podem ser criados pela interface web do GitHub.
- Nenhuma chave da NASA: a API POWER é aberta.
- Conhecimento básico de navegação em sites; os comandos e códigos estão prontos para copiar.

Estrutura final do repositório

```
POWER/
|-- index.html
|-- app.js
|-- config.js
|-- README.md
|-- scripts/
|   `-- fetch_nasa.py
|-- sql/
|   `-- setup.sql
|-- .github/
|   `-- workflows/
|       `-- update-data.yml
`-- docs/
    |-- tutorial.pdf
    |-- ai-interaction.md
    |-- reflexao.md
    `-- apresentacao.pdf
```

4. Etapa 1 - GitHub: organização e repositório

1. No GitHub, clique no ícone **+** no topo e escolha **New organization**. Selecione o plano gratuito.
2. Dê à organização o nome `SN-2026-GRUPO-Nº-NASA`, trocando Nº pelo número do grupo com dois dígitos (exemplo: `SN-2026-GRUPO-15-NASA`).
3. Dentro da organização, crie um repositório **público** chamado `POWER` e marque a opção de criar com um README.
4. Em **People > Invite member**, convide o usuário `GilFerques` (professor avaliador).

Atenção: o repositório precisa ser público. Além de ser exigência da atividade, o GitHub Pages gratuito só funciona em repositórios públicos. Se o repositório foi criado em uma conta pessoal, use Settings > General > Danger Zone > Transfer ownership para movê-lo para a organização e, depois, renomeie-o para POWER.

5. Etapa 2 - Supabase: organização e projeto

1. Em supabase.com, crie uma **Organization** com o mesmo nome usado no GitHub (exemplo: `SN-2026-GRUPO-15-NASA`).
2. Dentro dela, clique em **New project** e use o nome `POWER-DB`.
3. Escolha a região **South America (São Paulo)** e defina uma senha forte para o banco (guarde-a em local seguro).
4. Aguarde alguns minutos até o projeto ficar ativo.
5. Convide o email `rafael.ferques@ifpr.edu.br` com a permissão **Developer** (Settings > Team > Invite).

Se o Supabase avisar que o limite de projetos gratuitos foi atingido, pause ou exclua um projeto antigo de atividades anteriores.

6. Etapa 3 - Banco de dados

No Supabase, abra **SQL Editor > New query**, cole o conteúdo abaixo e clique em **Run**. Salve também esse mesmo texto no repositório, no arquivo `sql/setup.sql`.

sql/setup.sql

```
create table if not exists public.clima_diario (
  id bigint generated always as identity primary key,
  local text not null,
  latitude numeric(8,4) not null,
  longitude numeric(8,4) not null,
  data date not null,
  temp_media numeric,
  precipitacao numeric,
  radiacao_solar numeric,
  vento numeric,
  umidade numeric,
  criado_em timestamptz default now(),
  constraint clima_diario_local_data_key unique (local, data)
);

create table if not exists public.execucoes (
  id bigint generated always as identity primary key,
  executado_em timestamptz default now(),
  registros_processados int default 0,
  lotes int default 0,
  erros int default 0,
  status text check (status in ('concluido','erro_parcial','erro_critico')),
  mensagem text
);

alter table public.clima_diario enable row level security;
alter table public.execucoes enable row level security;

create policy "leitura publica" on public.clima_diario
  for select using (true);
create policy "leitura publica" on public.execucoes
  for select using (true);

grant select on public.clima_diario, public.execucoes
  to anon, authenticated;
grant all on public.clima_diario, public.execucoes
  to service_role;
grant usage, select on all sequences in schema public
  to service_role;
```

O que cada parte faz

- **unique (local, data):** cada linha representa uma cidade em um dia. Essa chave natural permite o upsert (atualizar se já existe, inserir se não existe), evitando duplicatas ao executar

o pipeline várias vezes.

- **Tabela execucoes:** log de cada execução, com registros processados, lotes, erros e status (concluido, erro_parcial ou erro_critico).
- **Row Level Security (RLS):** bloqueia o acesso às linhas por padrão. A política de leitura pública libera apenas consultas SELECT.
- **GRANTs:** SELECT para as funções anon e authenticated (leitura pelo painel) e ALL para service_role (escrita pelo pipeline).

Para conferir, abra **Table Editor** e verifique se aparecem as tabelas `clima_diario` e `execucoes`, ambas com a indicação de RLS ativo.

7. Etapa 4 - Credenciais do Supabase

Você precisará de três valores. Eles estão em **Integrations > Data API** (a URL) e em **Project Settings > API Keys** (as chaves).

Valor	Formato	Onde usar	É público?
Project URL	https://ID.supabase.co	Secret SUPABASE_URL e config.js	Sim
Publishable key	sb_publishable_...	config.js (painel)	Sim, somente leitura pelo RLS
Secret key	sb_secret_...	Secret SUPABASE_SERVICE_KEY	**Não**: nunca no código

Cuidado com a URL: a tela Data API mostra o endereço terminando em /rest/v1/. No secret e no config.js use apenas a base, `https://ID.supabase.co`, sem barra final. O código já acrescenta /rest/v1/ sozinho; deixá-lo no secret causa erro 404.

Se a secret key aparecer em algum arquivo, commit, print ou mensagem, gere outra em **New secret key**, apague a antiga e atualize o secret no GitHub.

8. Etapa 5 - Secrets no GitHub

1. No repositório, abra **Settings > Secrets and variables > Actions**.
2. Clique em **New repository secret**, use o nome SUPABASE_URL e cole a Project URL (somente https://ID.supabase.co, sem espaços ou quebras de linha).
3. Crie um segundo secret chamado SUPABASE_SERVICE_KEY e cole a secret key (sb_secret_...).

Como a NASA POWER não exige chave, não é necessário criar o secret NASA_API_KEY.

9. Etapa 6 - Script de coleta

No repositório, use **Add file > Create new file**, digite `scripts/fetch_nasa.py` no campo do nome (as barras criam as pastas) e cole o código abaixo.

scripts/fetch_nasa.py

```
import os
import sys
import time
import requests
from datetime import date, timedelta

SB_URL = os.environ["SUPABASE_URL"].strip().rstrip("/")
SB_KEY = os.environ["SUPABASE_SERVICE_KEY"].strip()

HEADERS = {"apikey": SB_KEY, "Content-Type": "application/json"}
if SB_KEY.startswith("eyJ"):
    HEADERS["Authorization"] = f"Bearer {SB_KEY}"

LOTE = 200
DIAS_HISTORICO = 90
DEFASAGEM_DIAS = 3

LOCAIS = [
    ("Marechal Cândido Rondon", -24.55, -54.06),
    ("Curitiba", -25.43, -49.27),
    ("Manaus", -3.12, -60.02),
    ("Porto Alegre", -30.03, -51.23),
    ("Recife", -8.05, -34.88),
    ("Brasília", -15.78, -47.93),
]

PARAMS = "T2M,PRECTOTCORR,ALLSKY_SFC_SW_DWN,WS2M,RH2M"
POWER_URL = "https://power.larc.nasa.gov/api/temporal/daily/point"


def val(v):
    return None if v is None or v == -999 else v


def buscar(lat, lon):
    fim = date.today() - timedelta(days=DEFASAGEM_DIAS)
    ini = fim - timedelta(days=DIAS_HISTORICO)
    params = {
        "parameters": PARAMS,
        "community": "RE",
        "latitude": lat,
        "longitude": lon,
        "start": ini.strftime("%Y%m%d"),
        "end": fim.strftime("%Y%m%d"),
        "format": "JSON",
    }
    ultimo_erro = None
    for tentativa in range(1, 4):
        try:
            r = requests.get(POWER_URL, params=params, timeout=90)
            r.raise_for_status()
            return r.json()["properties"]["parameter"]
```

```
        except Exception as ex:
            ultimo_erro = ex
            print(f"  tentativa {tentativa}/3 falhou: {ex}")
            time.sleep(5 * tentativa)
    raise ultimo_erro


def normalizar(local, lat, lon, p):
    linhas = []
    for d in p["T2M"].keys():
        linhas.append({
            "local": local,
            "latitude": lat,
            "longitude": lon,
            "data": f"{d[:4]}-{d[4:6]}-{d[6:]}",
            "temp_media": val(p["T2M"].get(d)),
            "precipitacao": val(p["PRECTOTCORR"].get(d)),
            "radiacao_solar": val(p["ALLSKY_SFC_SW_DWN"].get(d)),
            "vento": val(p["WS2M"].get(d)),
            "umidade": val(p["RH2M"].get(d)),
        })
    return linhas


def registrar(proc, lotes, erros, status, msg=""):
    try:
        r = requests.post(
            f"{SB_URL}/rest/v1/execucoes",
            headers={**HEADERS, "Prefer": "return=minimal"},
            json={
                "registros_processados": proc,
                "lotes": lotes,
                "erros": erros,
                "status": status,
                "mensagem": msg[:500],
            },
            timeout=30,
        )
        if not r.ok:
            print(f"Falha ao registrar execução: {r.status_code} {r.text}")
    except Exception as ex:
        print(f"Falha ao registrar execução: {ex}")


def main():
    proc = lotes = erros = 0
    mensagens = []
    unicos = {}

    for local, lat, lon in LOCAIS:
        print(f"Buscando {local}...")
        try:
```

```
            for linha in normalizar(local, lat, lon, buscar(lat, lon)):
                unicos[(linha["local"], linha["data"])] = linha
        except Exception as ex:
            erros += 1
            mensagens.append(f"{local}: {ex}")
            print(f"Falha em {local}: {ex}")

    dados = list(unicos.values())
    print(f"{len(dados)} registros únicos para enviar.")

    if not dados:
        registrar(0, 0, erros or 1, "erro_critico",
                  "; ".join(mensagens) or "Nenhum dado coletado")
        sys.exit(1)

    for i in range(0, len(dados), LOTE):
        lote = dados[i:i + LOTE]
        r = requests.post(
            f"{SB_URL}/rest/v1/clima_diario?on_conflict=local,data",
            headers={**HEADERS,
                     "Prefer": "resolution=merge-duplicates,return=minimal"},
            json=lote,
            timeout=60,
        )
        if r.ok:
            proc += len(lote)
            lotes += 1
        else:
            erros += 1
            mensagens.append(f"lote {i // LOTE + 1}: {r.status_code}")
            print(f"Erro no lote {i // LOTE + 1}: {r.status_code} {r.text}")

    status = "concluido" if erros == 0 else "erro_parcial"
    registrar(proc, lotes, erros, status, "; ".join(mensagens))
    print(f"Fim: {proc} registros, {lotes} lotes, {erros} erros, "
          f"status={status}")

    if erros:
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Como o script funciona

- **val():** converte o valor -999 (dado ausente da POWER) em None, que vira NULL no banco.
- **buscar():** chama a API para uma coordenada, com até três tentativas e espera crescente entre elas.
- **normalizar():** transforma a resposta da API em linhas no formato da tabela clima_diario.
- **Deduplicação:** o dicionário `unicos`, indexado por (local, data), elimina repetições antes do envio, evitando o erro PostgreSQL 21000 (não é possível alterar a mesma linha duas vezes no mesmo comando de upsert).
- **Envio em lotes:** os dados vão em lotes de 200 registros, com `on_conflict=local,data` para fazer o upsert.
- **registrar():** grava na tabela execucoes o total de registros, lotes, erros e o status.
- **sys.exit(1):** se houve erro, o script termina com falha, e o workflow fica vermelho e visível nos logs.

Para monitorar outras cidades, basta incluir uma linha com nome, latitude e longitude na lista LOCAIS. Para mudar a janela de dados, altere DIAS_HISTORICO.

Observação: o cabeçalho Authorization só é enviado para chaves antigas no formato JWT (que começam com eyJ). As chaves novas (sb_secret_) usam apenas o cabeçalho apikey.

10. Etapa 7 - Workflow do GitHub Actions

Crie o arquivo `.github/workflows/update-data.yml` (com o ponto no início de .github) e cole o conteúdo abaixo.

.github/workflows/update-data.yml

```
name: Atualizar dados

on:
  schedule:
    - cron: "0 6 * * *"
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: update-data
  cancel-in-progress: false

jobs:
  coleta:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - name: Baixar o repositório
        uses: actions/checkout@v4

      - name: Configurar o Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Instalar dependências
        run: pip install requests

      - name: Executar o pipeline
        run: python scripts/fetch_nasa.py
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_SERVICE_KEY: ${{ secrets.SUPABASE_SERVICE_KEY }}
```

- **schedule (cron):** execução diária às 06:00 UTC.
- **workflow_dispatch:** cria o botão Run workflow para execução manual.
- **permissions: contents: read:** o workflow só pode ler o repositório (menor privilégio).
- **concurrency:** impede duas execuções simultâneas.
- **timeout-minutes:** encerra o job se passar de 10 minutos.
- **env:** entrega os Secrets ao script, que os lê com os.environ.

11. Etapa 8 - Executar e validar o pipeline

1. No repositório, abra a aba **Actions** (na barra do topo, ao lado de Pull requests e Projects).
2. Na lista da esquerda, clique em **Atualizar dados** e depois em **Run workflow > Run workflow**.
3. Aguarde de 1 a 2 minutos e atualize a página. O ícone deve ficar verde. Se ficar vermelho, abra o job **coleta** e leia o log do passo que falhou.

4. No Supabase, abra **Table Editor > clima_diario**: devem aparecer cerca de 546 linhas (6 cidades com 91 dias cada).
5. Abra **execucoes**: deve haver uma linha com status `concluido`.

Testar a API REST pública

Cole o endereço abaixo no navegador (troque o ID e a publishable key). O resultado deve ser um JSON com três registros:

```
https://SEU-ID-DO-PROJETO.supabase.co/rest/v1/clima_diario?select=*&limit=3&apikey=sb_publishable_SUA_CHAVE_PUBLICA
```

Resposta	Causa provável
JSON com registros	Tudo certo.
No API key found in request	Faltou a chave depois de apikey=.
Lista vazia []	Tabela vazia, ou política/GRANT de leitura ausente.
Erro 401 ou de permissão	Chave incorreta ou incompleta.
Tabela não encontrada	Nome da tabela diferente de clima_diario ou setup.sql não executado.

12. Etapa 9 - Painel web e GitHub Pages

Arquivos do painel

O painel é formado por três arquivos na raiz do repositório:

- `config.js`: guarda a URL do projeto e a publishable key (conteúdo abaixo);
- `index.html`: estrutura da página e estilos;
- `app.js`: busca os dados no Supabase, aplica os filtros e desenha os gráficos com a biblioteca Chart.js.

config.js

```
const SUPABASE_URL = "https://SEU-ID-DO-PROJETO.supabase.co";
const SUPABASE_KEY = "sb_publishable_SUA_CHAVE_PUBLICA";
```

Os arquivos `index.html` e `app.js` completos estão no repositório do projeto (https://github.com/SN-2026-GRUPO-15-NASA/POWER); copie-os para a raiz do seu repositório, mantendo a ordem dos scripts no final do HTML (config.js antes de app.js).

Recursos do painel

- Seleção de uma ou várias cidades, cada uma com uma cor própria;
- Abas de variável (temperatura, chuva, radiação solar, vento e umidade) e de período (7, 30, 60 dias ou tudo);
- Cards com média, mínimo, máximo e acumulados;
- Gráfico diário comparando as cidades selecionadas e gráfico de ranking clicável;
- Tabela ordenável pelo clique no título da coluna;
- Indicação da última atualização do pipeline, lida da tabela execucoes;
- Função escapeHtml() aplicada a todo valor externo inserido no HTML, evitando injeção de código (XSS).

Publicar no GitHub Pages

1. No repositório, abra **Settings > Pages**.
2. Em Build and deployment, escolha **Deploy from a branch**, selecione a branch **main** e a pasta **/ (root)**, e clique em **Save**.
3. Aguarde de 1 a 2 minutos. O endereço aparece no topo da página, no formato `https://organizacao.github.io/POWER/` (tudo em minúsculas).
4. Abra o link e pressione Ctrl+F5 para ignorar o cache do navegador.

Se o Pages pedir upgrade ou não permitir escolher a branch, o repositório está privado. Torne-o público em Settings > General > Danger Zone > Change visibility.

13. Segurança

- A **secret key** (service_role) fica somente nos Secrets do GitHub. Nunca a coloque em arquivos, commits, prints ou mensagens.
- O painel usa apenas a **publishable key**, que só consegue ler porque o RLS e os GRANTs limitam o acesso.
- O workflow usa `permissions: contents: read`, o mínimo necessário.
- Todo texto vindo do banco passa por `escapeHtml()` antes de entrar no HTML.
- Em caso de exposição de uma chave, gere uma nova imediatamente, apague a antiga e atualize o Secret.

14. Problemas comuns e soluções

Sintoma	Causa provável	Solução
Workflow vermelho com erro 401 ou 403	Secret key ou URL incorretas, ou com espaço no final.	Recriar os dois Secrets, copiando os valores direto do Supabase.
relation does not exist	O setup.sql não foi executado.	Rodar o SQL no SQL Editor do Supabase.
Erro 404 nas chamadas	SUPABASE_URL termina em /rest/v1/.	Deixar somente https://ID.supabase.co.
Erro 21000 no upsert	Registros duplicados no mesmo lote.	Manter a deduplicação por (local, data) antes do envio.
Erro no on_conflict	Constraint UNIQUE (local, data) ausente.	Recriar a tabela com o setup.sql.
Workflow não aparece na aba Actions	Arquivo em caminho incorreto.	Conferir o caminho .github/workflows/update-data.yml na branch main.
Painel com Carregando... para sempre	Erro de JavaScript (arquivo cortado ao colar) ou chave incorreta.	Abrir o Console (F12), ler o erro e recolar o arquivo inteiro.
Pages indisponível ou pedindo upgrade	Repositório privado.	Tornar o repositório público.
Radiação aparece como - em dias recentes	A NASA ainda não publicou o valor (-999).	Normal: o dado é completado em execuções futuras.

15. Checklist final

- Organização e repositório com os nomes no padrão da atividade; repositório público.
- Professor convidado no GitHub (GilFerques) e no Supabase (Developer).
- sql/setup.sql com UNIQUE, RLS e GRANTs executado no Supabase.
- Secrets SUPABASE_URL e SUPABASE_SERVICE_KEY configurados.
- Workflow com permissions, concurrency e timeout, e ao menos uma execução verde.
- Dados reais em clima_diario e registro em execucoes.
- Painel publicado no GitHub Pages, com filtro, última atualização e escapeHtml().
- Pasta docs/ com tutorial.pdf, apresentacao.pdf, ai-interaction.md e reflexao.md.
- Planilha de reserva atualizada com os links definitivos.

Referências

- NASA POWER: power.larc.nasa.gov
- Portal de APIs abertas da NASA: api.nasa.gov
- Supabase: supabase.com/docs
- GitHub Actions e GitHub Pages: docs.github.com